Rainfall Data Analysis and Nowcasting Using PyTorch and Google Earth Engine

A Mini Project Report submitted in partial fulfillment
of the requirements for the course PH 312

by

Ganesh Sinnur

230121021

Under the supervision of

Dr. Dilip Pal

PH312, Mini Project
Department of Physics
Indian Institute of Technology, Guwahati
781039 - Assam, India

April 2026

Indian Institute of Technology Guwahati

Guwahati – 781039, Assam, India

CERTIFICATE

It is certified that the work contained in this thesis entitled **"Rainfall Data Analysis and Nowcasting Using PyTorch and Google Earth Engine"** submitted by **Ganesh Sinnur (230121021)** in partial fulfilment of the requirements for the course PH 312 **Mini Project** in Engineering Physics at the Indian Institute of Technology Guwahati for the session 2023–2027 is an authentic work carried out by him under the supervision of Dr. Dilip Pal to the best of my knowledge and belief.

Signature

Dr. Dilip Pal
Professor
Department of Physics
Indian Institute of Technology Guwahati
Assam – 781039

Declaration

I hereby declare that the Mini Project report entitled "Rainfall Data Analysis and Nowcasting Using PyTorch and Google Earth Engine" submitted to the Indian Institute of Technology, Guwahati is a record of my original work. The content embodied in this project report has not been submitted to any other university for the award of any degree.

Ganesh Sinnur
230121021

Acknowledgement

First and foremost, I would like to express my deepest gratitude to my supervisor, Dr. Dilip Pal for his unwavering support, insightful guidance and invaluable encouragement throughout. Your expertise and dedication have been instrumental in shaping this project, and your constructive feedback has pushed me to reach new heights. I am truly fortunate to have had the opportunity to work under your supervision. Thank you for helping me turn my aspirations into reality.

Ganesh Sinnur
230121021

Abstract

Weather nowcasting—the prediction of atmospheric conditions in the immediate future (0–6 hours)—is a critical task for disaster preparedness, agriculture, and aviation. This mini-project presents a complete, end-to-end pipeline for rainfall nowcasting over the Indian subcontinent using deep learning. Multi-spectral satellite imagery from the MODIS Terra sensor and precipitation measurements from NASA's Global Precipitation Measurement (GPM) mission are acquired through Google Earth Engine (GEE). A stratified sampling strategy, combining precipitation intensity and terrain elevation bins, ensures that the training dataset contains balanced representation of diverse weather and geographical conditions across India. The sampled data is organized into 5×5 pixel patches with 7 spectral bands as inputs, paired with corresponding precipitation values at 2-hour and 6-hour forecast horizons as labels. A Convolutional Neural Network (CNN) built in PyTorch—featuring normalization, two convolutional layers, and a linear output head—is trained on this dataset using the Smooth L1 loss function and the Adam optimizer. The trained model successfully predicts future precipitation intensity from satellite imagery patches, demonstrating the viability of deep learning approaches for short-term rainfall forecasting over the Indian monsoon region.

Contents

Certificate **i**

Declaration **ii**

Acknowledgement **iii**

Abstract **iv**

1 Introduction **1**
1.1 Motivation and Background 1
1.2 Problem Statement 1
1.3 Overview of the Approach 1
1.4 Tools and Technologies 2

2 Data Sources and Collection **3**
2.1 Google Earth Engine 3
2.2 MODIS Terra — Input Satellite Imagery 3
2.3 GPM IMERG — Precipitation Labels 4
2.4 SRTM — Elevation Data 4
2.5 Region of Interest: India 4
2.6 Temporal Coverage 4

3 Creating the Dataset **6**
3.1 Stratified Sampling 6
3.2 Training Examples 7
3.2.1 Input Patch 7
3.2.2 Label Patch 7
3.3 Dataset Storage 7
3.4 Configuration Parameters 8

4 Model Architecture **9**
4.1 Normalization Layer 9
4.2 WeatherConfig 9
4.3 CNN Architecture 10
4.4 Loss Function: Smooth L1 Loss 11

5 Training the Model **12**
5.1 Data Loading and Preprocessing 12

5.2 Train/Test Split 12
5.3 Training Configuration 12
5.4 Training Loop 12
5.5 Model Saving 13

6 Results and Discussion **14**
6.1 Training Convergence 14
6.2 Predictions 14
6.3 Discussion 14
6.4 Limitations 14
6.5 Future Work 15

7 Conclusion **16**

A Python Source Code **18**

1 Introduction

1.1 Motivation and Background

Our model's objective is to forecast the weather using satellite imagery. We wish to forecast the amount of precipitation that will fall over the next two to six hours, expressed in millimetres per hour. Weather nowcasting is the word for this type of short-time forecasting.

Every image has a shape (width, height, and bands) when working with satellite data. Bands include numerical values for every pixel in the image, such as the probability of various classifications or measurements from certain satellite equipment for various electromagnetic spectrum bands.

Accurate rainfall prediction is especially important for India, where the monsoon season (June–September) brings the majority of annual rainfall and directly impacts agriculture, water resource management, and flood preparedness. Traditional Numerical Weather Prediction (NWP) models require enormous computational resources and often struggle with the fine spatial and temporal resolution needed for nowcasting. Data-driven approaches using deep learning offer a compelling alternative by learning complex, non-linear relationships directly from observational data.

1.2 Problem Statement

Given a multi-spectral satellite image patch of a region over India at a particular time, predict the precipitation intensity (in mm/h) for the same region at 2 hours and 6 hours into the future.

Formally, the task is to learn a mapping:

$$f : \mathbf{X} \in \mathbb{R}^{H \times W \times C} \rightarrow \mathbf{Y} \in \mathbb{R}^{H \times W \times 2} \tag{1.1}$$

where
$\mathbf{X}$ is the input satellite image patch with $H = W = 5$ pixels and $C = 7$ spectral bands
$\mathbf{Y}$ is the output precipitation map with 2 channels (forecast at +2h and +6h)

1.3 Overview of the Approach

The project follows a three-stage pipeline:

1. **Data Collection:** Satellite imagery and precipitation data are downloaded from Google Earth Engine using automated scripts.

2. **Dataset Creation:** A balanced training dataset is constructed using stratified sampling across precipitation and elevation bins.

3. **Model Training and Evaluation:** A CNN is trained on the dataset and evaluated on a held-out test set.

1.4 Tools and Technologies

The following tools and libraries are used in this project:

Tool	**Purpose**
PyTorch	Deep learning framework for building and training the CNN
HuggingFace Transformers	Model serialization (save/load via `PreTrainedModel`)
Google Earth Engine (GEE)	Cloud platform for accessing satellite imagery
NumPy	Numerical array operations
Matplotlib	Plotting and visualization
python-dotenv	Environment variable management

Table 1.1: Tools and libraries used in the project.

2 Data Sources and Collection

This chapter describes the satellite data sources used in the project and how they are accessed through Google Earth Engine.

2.1 Google Earth Engine

Google Earth Engine (GEE) is a cloud-based geospatial analysis platform that provides access to petabytes of satellite imagery and geospatial datasets. It allows users to perform large-scale analysis directly on Google's infrastructure without needing to download raw data. In this project, GEE is used to fetch satellite imagery and precipitation measurements for specific dates and regions programmatically through its Python API (`earthengine-api`).

Authentication is performed using:

```
earthengine authenticate
```

and the project is initialized with a Google Cloud Project ID stored in a `.env` file.

2.2 MODIS Terra — Input Satellite Imagery

The Moderate Resolution Imaging Spectroradiometer (MODIS) is an instrument aboard NASA's Terra satellite, which has been operational since December 1999. MODIS provides daily global coverage with 36 spectral bands at varying spatial resolutions.

In this project, we use the **MODIS Terra Daily Surface Reflectance** product (`MODIS/061/MOD09GA`) which provides atmospherically corrected surface reflectance values. We select the following 7 spectral bands:

Band	**Name**	**Wavelength (nm)**	**Type**
Band 1	sur_refl_b01	620–670	Red (Visible)
Band 2	sur_refl_b02	841–876	Near Infrared (NIR)
Band 3	sur_refl_b03	459–479	Blue (Visible)
Band 4	sur_refl_b04	545–565	Green (Visible)
Band 5	sur_refl_b05	1230–1250	Shortwave Infrared (SWIR-1)
Band 6	sur_refl_b06	1628–1652	Shortwave Infrared (SWIR-2)
Band 7	sur_refl_b07	2105–2155	Shortwave Infrared (SWIR-3)

Table 2.1: MODIS Terra spectral bands used as model inputs.

MODIS was chosen because it provides **global coverage** (including India), daily revisit times, and multi-spectral data that captures atmospheric and surface properties relevant to weather prediction.

2.3 GPM IMERG — Precipitation Labels

The Global Precipitation Measurement (GPM) mission, led by NASA and JAXA, provides global precipitation estimates. We use the **IMERG V07** (Integrated Multi-satellite Retrievals for GPM) half-hourly product available as `NASA/GPM_L3/IMERG_V07` on GEE.

This dataset provides the `precipitation` band, which gives the estimated precipitation rate in mm/h at a spatial resolution of approximately 10 km. We use this as our **ground truth label** for training. Precipitation values are clamped to the range $[0, 30]$ mm/h to avoid extreme outliers:

$$p_{\text{clamped}} = \min(\max(p, 0), 30) \tag{2.1}$$

2.4 SRTM — Elevation Data

The Shuttle Radar Topography Mission (SRTM) dataset (`USGS/SRTMGL1_003`) provides global elevation data at approximately 30-meter resolution. Elevation is used during the **stratified sampling** phase to ensure that the training data includes points from diverse terrain—from coastal plains to the Himalayas. Elevation values are clamped to $[0, 6000]$ meters.

2.5 Region of Interest: India

Since MODIS provides global coverage, we restrict sampling to the **Indian subcontinent** using the following bounding rectangle:

$$\text{Region} = \text{Rectangle}([68°\text{E},\ 6°\text{N},\ 97°\text{E},\ 37°\text{N}]) \tag{2.2}$$

This covers mainland India from the southern tip of Tamil Nadu to the northern reaches of Kashmir, and from the western coast of Gujarat to the eastern borders of Arunachal Pradesh.

2.6 Temporal Coverage

Data is sampled from the **Indian monsoon season of 2023** (July–September), when rainfall activity is highest. The following dates are used:

Date	**Rationale**
2023-07-01	Start of peak monsoon
2023-07-15	Mid-July heavy rainfall
2023-08-01	Peak monsoon month
2023-08-15	Continued heavy rains
2023-09-01	Retreating monsoon

Table 2.2: Sample dates used for dataset creation during the Indian monsoon season.

3 Creating the Dataset

A dataset consists of training examples, which are (inputs, labels) pairs, so for each input data, we must give it the correct output values. This chapter explains how the training dataset is constructed.

3.1 Stratified Sampling

We want to get balanced points for a given time. We utilise `ee.Image.stratifiedSample` to select about the same number of spots for each amount of precipitation. Additionally, the majority of the areas from which we are choosing data points are located close to sea level, at extremely low heights.

Since the precipitation is a continuous value, we first need to convert it to a classification. By looking at different images, we noticed that most values fall within 0 and 30. So we simply clamped the values into that range, divided by the maximum value, multiplied by the number of bins, and converted them into integers.

We do a similar thing for the elevation, where we found empirically that most values fall between 0 and 6000. Once we have bins for both precipitation and elevation, we combine them into a single "unique" bin value to make sure we get all the possible precipitation values for each elevation.

So, we defined a function called `sample_points` that gives us a balanced selection of (longitude, latitude) coordinates for a given date.

```
precipitation_bins = (
    data.select('precipitation')
    .clamp(0, MAX_PRECIPITATION)
    .divide(MAX_PRECIPITATION)
    .multiply(num_bins - 1)
    .uint8()
)
```

Listing 3.1: Precipitation binning logic.

```
elevation_bins = (
    get_elevation()
    .clamp(0, MAX_ELEVATION)
    .divide(MAX_ELEVATION)
    .multiply(num_bins - 1)
    .uint8()
)
```

Listing 3.2: Elevation binning logic.

The combined unique bin is computed as:

$$\text{unique_bin} = (\text{elevation_bin} \times N_{\text{bins}}) + \text{precipitation_bin} \tag{3.1}$$

where $N_{\text{bins}} = 10$, creating $10 \times 10 = 100$ unique classes. Points are then sampled equally from each class using `ee.Image.stratifiedSample`.

3.2 Training Examples

We predefined that all our training examples would be 5 pixels width by 5 pixels height, but we could choose any size as long as the model accepts it. We also want all the training examples to be the same size so we can batch them.

We defined `get_training_example` in `create_dataset.py`, which retrieves a pair of (inputs, labels) for the specified date and (longitude, latitude) coordinate.

3.2.1 Input Patch

For each sampled point, a 5×5 pixel region is extracted from the MODIS satellite image. Each pixel covers approximately 5 km of ground (based on the sampling scale of 5000 meters). With 7 spectral bands, the input tensor has shape:

$$\mathbf{X} \in \mathbb{R}^{5\times5\times7} \tag{3.2}$$

3.2.2 Label Patch

The corresponding labels are obtained from GPM precipitation data at two future time steps:

- **+2 hours:** Precipitation at 2 hours after the input time
- **+6 hours:** Precipitation at 6 hours after the input time

These are extracted for the same 5×5 region, giving a label tensor of shape:

$$\mathbf{Y} \in \mathbb{R}^{5\times5\times2} \tag{3.3}$$

This is basically how the dataset was created.

3.3 Dataset Storage

All collected examples are saved as a compressed NumPy file:

`./data/weather_dataset.npz`

The file contains two arrays:

Key	Shape	Description
`inputs`	$(N, 5, 5, 7)$	N satellite image patches with 7 spectral bands
`labels`	$(N, 5, 5, 2)$	N precipitation grids at +2h and +6h

Table 3.1: Contents of the saved `weather_dataset.npz` file.

3.4 Configuration Parameters

Parameter	Value	Meaning
`NUM_BINS`	10	Number of bins for precipitation and elevation
`PATCH_SIZE`	5	5×5 pixel patches
`NUM_POINTS`	500	Target sample points per date
`SCALE`	5000	Meters per pixel (5 km resolution)
`max_points_per_date`	20	Maximum examples collected per date
`MAX_PRECIPITATION`	30	Clamp precipitation to 0–30 mm/h
`MAX_ELEVATION`	6000	Clamp elevation to 0–6000 meters

Table 3.2: Configuration parameters for dataset creation.

4 Model Architecture

The architecture used to train the model is shown in this chapter. The `transformers` package of PyTorch is used. From it we use the `PretrainedConfig` and `PreTrainedModel` classes. To split our dataset into training and a testing/validation subset, we use `Dataset.train_test_split` functions. In our trainer file we defined the `read_dataset` function to load our data files, and it returns us a Dataset with train/test splits.

4.1 Normalization Layer

The input data can consist of potentially very large numbers, but machine learning generally prefers small numbers around -1 and 1. So in our model we defined a Normalization layer which applies Z-Score to normalize all the model's inputs as a first step. But we need to provide it with the mean and standard deviation from the training dataset for the normalization layer.

The Z-Score normalization is defined as:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma} \tag{4.1}$$

where
x is the raw input value
μ is the per-band mean computed across the training set
σ is the per-band standard deviation

```
class Normalization(nn.Module):
    def __init__(self, mean, std):
        super().__init__()
        self.mean = nn.Parameter(
            torch.as_tensor(mean), requires_grad=False)
        self.std = nn.Parameter(
            torch.as_tensor(std), requires_grad=False)

    def forward(self, x):
        return (x - self.mean) / self.std
```

Listing 4.1: Normalization layer implementation.

4.2 WeatherConfig

The `WeatherConfig` contains all the model's hyperparameters, and we must also pass the mean and standard deviation from the training dataset for the normalization layer.

```
class WeatherConfig(PretrainedConfig):
```

```python
    model_type = "weather"

    def __init__(
        self,
        num_inputs=7,
        num_hidden1=64,
        num_hidden2=128,
        num_outputs=2,
        kernel_size=(3, 3),
        mean=[0.0] * 7,
        std=[1.0] * 7,
        **kwargs
    ):
        super().__init__(**kwargs)
        self.num_inputs = num_inputs
        self.num_hidden1 = num_hidden1
        self.num_hidden2 = num_hidden2
        self.num_outputs = num_outputs
        self.kernel_size = kernel_size
        self.mean = mean
        self.std = std
```

Listing 4.2: Model configuration class.

4.3 CNN Architecture

The WeatherModel class inherits from `PreTrainedModel` to make it compatible with HuggingFace Transformers. The model definition includes the loss function, so it knows how good or bad their predictions were. We could use any regression loss function like Mean Absolute Error (L1) or Mean Squared Error (L2). PyTorch provides a Smooth L1 Loss, which chooses between L1 and L2 depending on a certain criteria. It's less sensitive to outliers, so we'll use that. To create a WeatherModel, we have to pass it a WeatherConfig.

The architecture consists of the following layers:

1. **Normalization**: Z-Score normalization using training set statistics
2. **MoveDim**: Converts from channels-last (B, H, W, C) to channels-first (B, C, H, W) format required by PyTorch's `Conv2d`
3. **Conv2D Layer 1**: $C_{\text{in}} = 7 \rightarrow C_{\text{out}} = 64$ filters with 3×3 kernel and `same` padding
4. **ReLU Activation**: Sets negative values to zero
5. **Conv2D Layer 2**: $C_{\text{in}} = 64 \rightarrow C_{\text{out}} = 128$ filters with 3×3 kernel and `same` padding
6. **ReLU Activation**
7. **MoveDim**: Converts back to channels-last format

8. **Linear Layer**: 128 → 2 (predicts precipitation at +2h and +6h per pixel)

9. **ReLU Activation**: Ensures precipitation predictions are non-negative

```python
class WeatherModel(PreTrainedModel):
    config_class = WeatherConfig

    def __init__(self, config):
        super().__init__(config)
        self.layers = nn.Sequential(
            Normalization(config.mean, config.std),
            MoveDim(-1, 1),
            nn.Conv2d(config.num_inputs, config.num_hidden1,
                      config.kernel_size, padding='same'),
            nn.ReLU(),
            nn.Conv2d(config.num_hidden1, config.num_hidden2,
                      config.kernel_size, padding='same'),
            nn.ReLU(),
            MoveDim(1, -1),
            nn.Linear(config.num_hidden2, config.num_outputs),
            nn.ReLU()
        )
        self.loss_fn = nn.SmoothL1Loss()

    def forward(self, inputs, labels=None):
        predictions = self.layers(inputs)
        loss = None
        if labels is not None:
            loss = self.loss_fn(predictions, labels)
        return {"loss": loss, "logits": predictions}
```

Listing 4.3: WeatherModel forward pass and loss computation.

4.4 Loss Function: Smooth L1 Loss

The Smooth L1 Loss (also called Huber Loss) is defined as:

$$\text{SmoothL1}(x, y) = \begin{cases} 0.5(x-y)^2 & \text{if } |x-y| < 1 \\ |x-y| - 0.5 & \text{otherwise} \end{cases} \tag{4.2}$$

where
x is the predicted value
y is the ground truth value

This loss function behaves like Mean Squared Error for small differences (providing smooth gradients) and like Mean Absolute Error for large differences (being robust to outliers). This is particularly useful for precipitation prediction, where heavy rainfall events can produce large values.

5 Training the Model

5.1 Data Loading and Preprocessing

The dataset is loaded from the `weather_dataset.npz` file. The `inputs` and `labels` arrays are converted to PyTorch tensors of type `float32`. The per-band mean and standard deviation are computed across all samples and spatial dimensions:

$$\mu_c = \frac{1}{N \cdot H \cdot W} \sum_{n,i,j} X_{n,i,j,c}, \quad \sigma_c = \sqrt{\frac{1}{N \cdot H \cdot W} \sum_{n,i,j} (X_{n,i,j,c} - \mu_c)^2} \tag{5.1}$$

where c indexes the spectral band and N, H, W are the number of samples, height, and width respectively.

5.2 Train/Test Split

The dataset is split into training and test sets using an 80/20 ratio:

- **Training set (80%):** Used to update the model's weights through backpropagation
- **Test set (20%):** Used to evaluate generalization performance on unseen data

5.3 Training Configuration

Hyperparameter	**Value**
Batch Size	32
Learning Rate	1×10^{-3}
Number of Epochs	50
Optimizer	Adam
Loss Function	Smooth L1 Loss
Train/Test Split	80% / 20%

Table 5.1: Training hyperparameters.

5.4 Training Loop

For each epoch, the following steps are performed:

1. The training data is shuffled and divided into mini-batches of size 32.

2. For each batch:
   (a) Forward pass: input patches are fed through the model to produce predictions.
   (b) Loss computation: Smooth L1 Loss is computed between predictions and ground truth labels.
   (c) Backward pass: gradients are computed via backpropagation.
   (d) Parameter update: the Adam optimizer adjusts the model's weights.
3. After each epoch, the model is evaluated on the test set (without gradient computation) to monitor generalization.

The Adam optimizer is used because it adapts the learning rate for each parameter individually, which typically leads to faster convergence compared to standard stochastic gradient descent.

5.5 Model Saving

After training, the model is saved using HuggingFace's `save_pretrained` method:

```
model.save_pretrained('./output/model.pt')
```

This creates a directory containing:

- `config.json`: The model configuration (architecture, mean/std values)
- `model.safetensors`: The trained weights

The model can later be reloaded using:

```
model = WeatherModel.from_pretrained('./output/model.pt')
```

6 Results and Discussion

6.1 Training Convergence

The training loss curve shows the Smooth L1 Loss decreasing over 50 epochs for both training and test sets. A consistently decreasing test loss indicates that the model is learning generalizable features rather than memorizing the training data.

6.2 Predictions

Here's what our output looks like against the ground truth labels. The model produces precipitation predictions at two forecast horizons (+2h and +6h). Visual comparison between predicted and actual precipitation maps demonstrates the model's ability to capture the spatial distribution of rainfall.

6.3 Discussion

The results demonstrate several key findings:

- **Feasibility of satellite-based nowcasting:** The CNN successfully learns a mapping from multi-spectral satellite imagery to future precipitation, validating the approach for the Indian monsoon region.
- **Importance of stratified sampling:** Without balanced sampling across precipitation and elevation bins, the model would be biased toward predicting "no rain" (since the majority of the Earth's surface is dry at any given time). The stratified sampling strategy ensures diverse training examples.
- **Role of normalization:** Z-Score normalization of the input bands is critical for stable training. Without it, bands with large absolute values (e.g., SWIR reflectance) would dominate the learning process.
- **Smooth L1 Loss advantage:** The Smooth L1 Loss provides robustness to outlier precipitation events (extreme rainfall), which would cause instability with standard MSE loss.

6.4 Limitations

- The current dataset uses a limited number of dates (5 days). A production model would require hundreds of dates spanning multiple monsoon seasons.

- MODIS provides only 1–2 overpasses per day, limiting temporal resolution compared to geostationary satellites like INSAT-3D.
- The 5×5 pixel patch size at 5 km resolution covers only a 25×25 km region, which may miss larger-scale weather patterns.
- The model does not incorporate temporal sequences (e.g., how clouds move over time), which could improve forecast accuracy.

6.5 Future Work

- Expand the dataset to include multiple years of monsoon data.
- Incorporate temporal sequences using recurrent architectures (LSTM/ConvLSTM).
- Use higher-resolution or geostationary satellite data (e.g., INSAT-3D) for more frequent observations over India.
- Deploy the trained model as a web service for real-time nowcasting.

7 Conclusion

This project demonstrates a complete end-to-end pipeline for rainfall nowcasting over India using deep learning. By leveraging freely available satellite data from Google Earth Engine—specifically MODIS Terra imagery for inputs and GPM IMERG for precipitation labels—we constructed a balanced training dataset covering the Indian monsoon season. A Convolutional Neural Network was designed, trained, and evaluated to predict precipitation intensity at 2-hour and 6-hour forecast horizons from multi-spectral satellite image patches.

The key contributions of this project are:

1. Implementation of a stratified sampling strategy that combines precipitation and elevation bins to create a balanced, representative dataset.

2. Design of a CNN architecture with built-in normalization, tailored for the regression task of precipitation prediction.

3. Successful training and evaluation of the model, demonstrating the viability of data-driven approaches for short-term rainfall forecasting over the Indian subcontinent.

The project establishes a foundation for more sophisticated nowcasting systems that could incorporate temporal dynamics, higher resolution data, and larger training datasets to achieve operational forecast accuracy.

References

[1] N. Gorelick, M. Hancher, M. Dixon, S. Ilyushchenko, D. Thau, and R. Moore, "Google Earth Engine: Planetary-scale geospatial analysis for everyone," *Remote Sensing of Environment*, vol. 202, pp. 18–27, 2017.

[2] G. J. Huffman et al., "Integrated Multi-satellite Retrievals for the Global Precipitation Measurement (GPM) Mission (IMERG)," *Advances in Global Change Research*, vol. 67, pp. 343–353, 2020.

[3] C. O. Justice et al., "The MODIS fire products," *Remote Sensing of Environment*, vol. 83, pp. 244–262, 2002.

[4] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[5] P. J. Huber, "Robust Estimation of a Location Parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.

[6] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[7] T. G. Farr et al., "The Shuttle Radar Topography Mission," *Reviews of Geophysics*, vol. 45, no. 2, 2007.

A Python Source Code

This appendix contains the core Python source files used in the project.

Weather Data Utilities (weather_data.py)

```python
import ee
import os
from dotenv import load_dotenv

load_dotenv()

MAX_PRECIPITATION = 30
MAX_ELEVATION = 6000
SCALE = 5000

def initialize_gee():
    project_id = os.getenv("GEE_PROJECT_ID")
    try:
        if project_id:
            ee.Initialize(project=project_id)
        else:
            ee.Initialize()
    except Exception as e:
        print(f"Error: {e}")
        raise

def get_gpm(date):
    if isinstance(date, str):
        date = ee.Date(date)
    start = date
    end = date.advance(1, 'day')
    gpm = (ee.ImageCollection('NASA/GPM_L3/IMERG_V07')
           .filterDate(start, end)
           .select('precipitation'))
    return gpm

def get_satellite_data(date):
    if isinstance(date, str):
        date = ee.Date(date)
    start = date
    end = date.advance(1, 'day')
    modis = (ee.ImageCollection('MODIS/061/MOD09GA')
             .filterDate(start, end)
```

```
            .select([
                'sur_refl_b01', 'sur_refl_b02',
                'sur_refl_b03', 'sur_refl_b04',
                'sur_refl_b05', 'sur_refl_b06',
                'sur_refl_b07',
            ]))
    return modis

def get_elevation():
    return ee.Image('USGS/SRTMGL1_003').select('elevation')

def get_precipitation_bins(data, num_bins):
    return (data.select('precipitation')
            .clamp(0, MAX_PRECIPITATION)
            .divide(MAX_PRECIPITATION)
            .multiply(num_bins - 1).uint8())

def get_elevation_bins(num_bins):
    return (get_elevation()
            .clamp(0, MAX_ELEVATION)
            .divide(MAX_ELEVATION)
            .multiply(num_bins - 1).uint8())
```

Listing A.1: Functions for GEE initialization and satellite data retrieval.

Model Definition (`models.py`)

```
import torch
import torch.nn as nn
from transformers import PretrainedConfig, PreTrainedModel

class Normalization(nn.Module):
    def __init__(self, mean, std):
        super().__init__()
        if not isinstance(mean, torch.Tensor):
            mean = torch.tensor(mean)
        if not isinstance(std, torch.Tensor):
            std = torch.tensor(std)
        self.mean = nn.Parameter(mean, requires_grad=False)
        self.std = nn.Parameter(std, requires_grad=False)
    def forward(self, x):
        return (x - self.mean) / self.std

class MoveDim(nn.Module):
    def __init__(self, source, destination):
        super().__init__()
        self.source = source
        self.destination = destination
```

```python
    def forward(self, x):
        return x.movedim(self.source, self.destination)

class WeatherConfig(PretrainedConfig):
    model_type = "weather"
    def __init__(self, num_inputs=7, num_hidden1=64,
                 num_hidden2=128, num_outputs=2,
                 kernel_size=(3, 3),
                 mean=[0.0]*7, std=[1.0]*7, **kwargs):
        super().__init__(**kwargs)
        self.num_inputs = num_inputs
        self.num_hidden1 = num_hidden1
        self.num_hidden2 = num_hidden2
        self.num_outputs = num_outputs
        self.kernel_size = kernel_size
        self.mean = mean
        self.std = std

class WeatherModel(PreTrainedModel):
    config_class = WeatherConfig
    def __init__(self, config):
        super().__init__(config)
        self.layers = nn.Sequential(
            Normalization(config.mean, config.std),
            MoveDim(-1, 1),
            nn.Conv2d(config.num_inputs, config.num_hidden1,
                      config.kernel_size, padding='same'),
            nn.ReLU(),
            nn.Conv2d(config.num_hidden1, config.num_hidden2,
                      config.kernel_size, padding='same'),
            nn.ReLU(),
            MoveDim(1, -1),
            nn.Linear(config.num_hidden2, config.num_outputs),
            nn.ReLU()
        )
        self.loss_fn = nn.SmoothL1Loss()

    def forward(self, inputs, labels=None):
        predictions = self.layers(inputs)
        loss = None
        if labels is not None:
            loss = self.loss_fn(predictions, labels)
        return {"loss": loss, "logits": predictions}
```

Listing A.2: WeatherConfig and WeatherModel definitions.